JAVA WITH DSA-DAY 8

PIVOT ELEMENT (RETURN INDEX):

```
class Main {
    public static void main(String[] args) {
        int n=6;
        int arr[]={1,4,5,8,7,3};
        int totalsum=0;
        for(int i=0;i<n;i++){
            totalsum+=arr[i];
        }
        int leftsum=0;
        for(int i=0;i<n;i++){
            int rightsum=totalsum-leftsum-arr[i];
            if(leftsum==rightsum){
                System.out.println(i);
                return;
            }
            leftsum+=arr[i];
        }
        System.out.println("-1");
    }
}
```

OUTPUT:

3

TIME COMPLEXITY

O(1):

```
import java.util.*;
public class Main {
```

```
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        System.out.println(arr[2]);
    }
}
```

O(n):

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        for(int i = 0; i < arr.length; i++) {
            System.out.println(arr[i]);
        }
    }
}
```

O(n^2):

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int n = 3;
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= n; j++) {
                System.out.println(i + " " + j);
            }
        }
    }
}
```

O(n^3):

```
public class Main {
```

```
    public static void main(String[] args) {
        int n = 3;
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= n; j++) {
                for(int k = 1; k <= n; k++) {
                    System.out.println(i + " " + j + " " + k);
                }
            }
        }
    }
}
```

O(n^3):Matrix:

```
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        for(int k = 0; k < n; k++) {
            result[i][j] += a[i][k] * b[k][j];
        }
    }
}
```

O(log n):

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50, 60, 70};
        int target = 50;
        int left = 0;
        int right = arr.length - 1;
        while(left <= right) {
            int mid = (left + right) / 2;
            if(arr[mid] == target) {
```

```
            System.out.println("Found");
            break;
        }
        else if(arr[mid] < target) {
            left = mid + 1;
        }
        else {
            right = mid - 1;
        }
    }
}
```

O(n log n):

```
import java.util.*;
public class Main {
    static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = (left + right) / 2;
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }
    static void merge(int[] arr, int left, int mid, int right) {
        System.out.println("Merging...");
    }
    public static void main(String[] args) {
        int[] arr = {5, 2, 8, 1};
        mergeSort(arr, 0, arr.length - 1);
```

```
    }
}
```

O(2^n):

```
public class Main {
    static int fib(int n) {
        if(n <= 1)
            return n;
        return fib(n - 1) + fib(n - 2);
    }
    public static void main(String[] args) {
        int n = 5;
        System.out.println(fib(n));
    }
}
```

O(n!):

```
public class Main {
    static void permute(String str, String ans) {
        if (str.length() == 0) {
            System.out.println(ans);
            return;
        }
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            String left = str.substring(0, i);
            String right = str.substring(i + 1);
            permute(left + right, ans + ch);
        }
    }
    public static void main(String[] args) {
```

```
        String str = "ABC";
        permute(str, "");
    }
}
```

COLLECTIONS

LIST:

ARRAYLIST

ArrayList Operations:

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        // add()
        list.add(10);
        list.add(20);
        list.add(30);
        list.add(10);
        System.out.println("After add(): " + list);
        // get()
        System.out.println("After getting the element at index 2: " + list.get(2));
        // set()
        list.set(1, 100);
        System.out.println("After set(): " + list);
        // remove() using index
        list.remove(2);
        System.out.println("After remove(index): " + list);
        // remove() using value
        list.remove(Integer.valueOf(40));
        System.out.println("After remove(value): " + list);
```

```
        // size()
        System.out.println("Size: " + list.size());
        // contains()
        System.out.println("Contains 100? " + list.contains(100));
        // isEmpty()
        System.out.println("Is Empty? " + list.isEmpty());
        // clear()
        list.clear();
        System.out.println("After clear(): " + list);
        System.out.println("Is Empty Now? " + list.isEmpty());
    }
}
```

OUTPUT:

```
After add(): [10, 20, 30, 10]
After getting the element at index 2: 30
After set(): [10, 100, 30, 10]
After remove(index): [10, 100, 10]
After remove(value): [10, 100, 10]
Size: 3
Contains 100? true
Is Empty? false
After clear(): []
Is Empty Now? True
```

DISPLAY EMPLOYEE DETAILS USING ARRAYLIST:

```
import java.util.*;
class Employee {
    int id;
    String name;
    String designation;
```

```
    Employee(int id, String name, String designation) {
        this.id = id;
        this.name = name;
        this.designation = designation;
    }
    void display() {
        System.out.println("Employee ID: " + id);
        System.out.println("Employee Name: " + name);
        System.out.println("Employee Designation: " + designation);
        System.out.println();
    }
}
public class Main {
    public static void main(String[] args) {
        ArrayList<Employee> list = new ArrayList<>();
        list.add(new Employee(101, "ABC", "HR"));
        list.add(new Employee(102, "DEF", "Project Manager"));
        list.add(new Employee(103, "GHI", "Accountant"));
        for(int i=0;i<list.size();i++){
            list.get(i).display();
        }
    }
}
```

OUTPUT:

```
Employee ID: 101
Employee Name: ABC
Employee Designation: HR

Employee ID: 102
```

Employee Name: DEF

Employee Designation: Project Manager

Employee ID: 103

Employee Name: GHI

Employee Designation: Accountant

Another method:

```
import java.util.*;
class Employee {
    int id;
    String name;
    String designation;
    Employee(int id, String name, String designation) {
        this.id = id;
        this.name = name;
        this.designation = designation;
    }
}
public class Main {
    public static void main(String[] args) {
        ArrayList<Employee> list = new ArrayList<>();
        Employee emp =new Employee(101, "ABC", "HR");
        Employee emp2 =new Employee(102, "DEF", "Project Manager");
        list.add(emp);
        list.add(emp2);
        for(int i = 0; i < list.size(); i++) {
            Employee e = list.get(i);
            System.out.println("ID: " + e.id);
            System.out.println("Name: " + e.name);
```

```
            System.out.println("Designation: " + e.designation);
            System.out.println();
        }
    }
}
```

OUTPUT:

ID: 101

Name: ABC

Designation: HR

ID: 102

Name: DEF

Designation: Project Manager

LINKEDLIST

SINGLY LINKED LIST:

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        LinkedList<Integer> list = new LinkedList<>();
        // add()
        list.add(10);
        list.add(20);
        list.add(30);
        System.out.println("After add(): " + list);
        // addFirst()
        list.addFirst(5);
        System.out.println("After addFirst(): " + list);
        // addLast()
        list.addLast(40);
```

```
System.out.println("After addLast(): " + list);
// get()
System.out.println("Element at index 2: " + list.get(2));
// getFirst()
System.out.println("First: "+ list.getFirst());
// getLast()
System.out.println("Last: " + list.getLast());
// set()
list.set(1, 100);
System.out.println("After set(): " + list);
// contains()
System.out.println("Contains 20? " + list.contains(20));
// size()
System.out.println("Size: " + list.size());
// remove(index)
list.remove(2);
System.out.println("After remove(index): "+ list);
// remove(value)
list.remove(Integer.valueOf(40));
System.out.println("After remove(value): " + list);
// removeFirst()
list.removeFirst();
System.out.println("After removeFirst(): "+ list);
// removeLast()
list.removeLast();
System.out.println("After removeLast(): " + list);
// isEmpty()
System.out.println("Is Empty? " + list.isEmpty());
// clear()
```

```
        list.clear();
        System.out.println("After clear(): "+ list);
        System.out.println("Is Empty Now? "+ list.isEmpty());
    }
}
```

OUTPUT:

```
After add(): [10, 20, 30]
After addFirst(): [5, 10, 20, 30]
After addLast(): [5, 10, 20, 30, 40]
Element at index 2: 20
First: 5
Last: 40
After set(): [5, 100, 20, 30, 40]
Contains 20? true
Size: 5
After remove(index): [5, 100, 30, 40]
After remove(value): [5, 100, 30]
After removeFirst(): [100, 30]
After removeLast(): [100]
Is Empty? false
After clear(): []
Is Empty Now? true
```

Without LinkedList<Integer> list = new LinkedList<> (); :

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
```

```
        this.next = null;
    }
}

public class Main {
    static Node head = null;
    // Insert at beginning
    static void insertFirst(int data) {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
    }

    // Insert at end
    static void insertLast(int data) {
        Node newNode = new Node(data);
        if(head == null) {
            head = newNode;
            return;
        }
        Node temp = head;
        while(temp.next != null) {
            temp = temp.next;
        }
        temp.next = newNode;
    }

    // Insert at position
    static void insertPosition(int data, int position) {
```

```
    Node newNode =new Node(data);
    if(position == 1) {
        insertFirst(data);
        return;
    }
    Node temp = head;
    for(int i=1; temp!=null && i<position-1; i++) {
        temp = temp.next;
    }
    if(temp == null) {
        System.out.println("Invalid Position");
        return;
    }
    newNode.next =temp.next;
    temp.next = newNode;
}

// Delete first
static void deleteFirst() {
    if(head != null)
        head = head.next;
}

// Delete last
static void deleteLast() {
    if(head == null)
        return;
    if(head.next == null) {
        head = null;
```

```
        return;
    }
    Node temp = head;
    while(temp.next.next != null) {
        temp = temp.next;
    }
    temp.next = null;
}

// Delete node
static void deleteNode(int value) {
    if(head == null)
        return;
    if(head.data == value) {
        head = head.next;
        return;
    }
    Node temp = head;
    while(temp.next != null &&temp.next.data != value) {
        temp = temp.next;
    }
    if(temp.next != null) {
        temp.next =temp.next.next;
    }
}

// Display
static void display() {
    Node temp = head;
```

```
        while(temp != null) {
            System.out.print( temp.data + " -> ");
            temp = temp.next;
        }
        System.out.println("null");
    }

    public static void main(String[] args) {
        insertFirst(20);
        insertFirst(10);
        insertLast(30);
        insertLast(40);
        System.out.println("Linked List:");
        display();
        insertPosition(25,3);
        System.out.println("After insertPosition:");
        display();
        deleteFirst();
        System.out.println("After deleteFirst:");
        display();
        deleteLast();
        System.out.println("After deleteLast:");
        display();
        deleteNode(25);
        System.out.println("After deleteNode:");
        display();
    }
}
```

OUTPUT:

Linked List:

10 -> 20 -> 30 -> 40 -> null

After insertPosition:

10 -> 20 -> 25 -> 30 -> 40 -> null

After deleteFirst:

20 -> 25 -> 30 -> 40 -> null

After deleteLast:

20 -> 25 -> 30 -> null

After deleteNode:

20 -> 30 -> null

STACK

VECTOR